DOCUMENTACIÓN TÉCNICA

Informe de arquitectura técnica — Inspira WMS

Análisis estático del código de la plataforma `inspira.wmslatam.com`: arquitectura, módulos propios, modelo de datos, integraciones, seguridad y mantenibilidad.

Fecha de elaboración	4 de noviembre de 2026
Objeto del análisis	Directorio `wp-content` de Inspira WMS — 591 MB, 61 plugins, 7 temas
Base del informe	Inspección estática de código fuente (solo lectura). No se modificó ningún archivo del proyecto.
Código propio analizado	≈ 33.000 líneas PHP en 12 unidades de desarrollo propio
Versión del documento	1.0

> **Alcance y límites.** El análisis se basa exclusivamente en los archivos presentes en `wp-content`. No se dispuso de acceso a la base de datos, a `wp-config.php`, a la configuración del servidor, al directorio `uploads` ni al historial de versiones. Las afirmaciones se clasifican a lo largo del documento como **hecho comprobado**, **inferencia** o **pendiente de verificación**. No se reproduce en este informe ningún token, secreto ni credencial: cuando existen, se cita únicamente su ubicación.

Índice

Las páginas indicadas corresponden a la primera página de cada sección. Cada sección comienza en página nueva.

1	Resumen general	4
2	Arquitectura del sistema	7
3	Estructura de archivos y carpetas	9
4	Funcionalidades y módulos	12
5	Base de datos	15
6	Flujos principales	19
7	Integraciones externas	21
8	Elementos propios de WordPress	24
9	Seguridad	28
10	Rendimiento y mantenimiento	32
11	Configuración y despliegue	35
12	Conclusiones y recomendaciones	38

Figuras

Figura 1 — Ecosistema WMS Latam: los tres sitios y sus servicios externos

Figura 2 — Capas internas de la instalación de Inspira

Figura 3 — Modelo de datos multiempresa

Figura 4 — Flujo de alta de usuario desde el panel de empresa

Figura 5 — Flujo del test, asignación de cursos e informe PDF

Figura 6 — Flujo de inicio de sesión único (SSO) entre sitios

Convenciones

Hecho comprobado	Verificado leyendo el archivo que se cita (ruta y línea).
Inferencia	Deducción razonada a partir del código, no confirmada por una fuente directa.
Pendiente de verificación	Requiere acceso a base de datos, servidor o entorno de ejecución.
Código propio	Desarrollo a medida para este ecosistema (no distribuido por terceros).

1 Resumen general

Inspira WMS es una plataforma de formación corporativa (B2B) construida sobre WordPress. Vende y entrega un recorrido que empieza en un test de diagnóstico y termina en un itinerario de cursos personalizado, gestionado por la empresa cliente y medido mediante informes.

1.1 Qué es el proyecto

El material analizado es el directorio `wp-content` completo de `inspira.wmslatam.com`. No se trata de una aplicación autónoma, sino de una instalación de WordPress fuertemente extendida: el valor del proyecto no está en el núcleo de WordPress sino en una capa propia de aproximadamente 33.000 líneas de PHP que implementa el modelo de negocio.

El propósito funcional, deducido de forma consistente del código, es el siguiente: una **empresa cliente** contrata un número de plazas; su administrador organiza esas plazas en **áreas** (departamentos o grupos) con cupo y fecha de caducidad; dentro de cada área da de alta a sus **empleados**; cada empleado realiza un **test** de diagnóstico cuyas respuestas determinan automáticamente qué **cursos de LearnDash** se le asignan; y el administrador de la empresa consulta el avance de su gente y descarga informes en PDF y Excel.

1.2 Funcionalidades principales

Tabla 1 — Capacidades funcionales identificadas y dónde están implementadas

Capacidad	Descripción	Implementación principal
Gestión multiempresa	CRUD de empresas (rol `empresa_admin`), áreas con cupo y fecha de expiración, y usuarios finales con rol `subscriber`.	`plugins/gestionar-empresa`
Panel del cliente	Vista de solo-su-empresa: áreas asignadas, listado de empleados, estado del test, progreso de cursos, informes PDF y exportación a Excel, recordatorios por correo.	`plugins/areas-asignadas`
Alta masiva de usuarios	Importación por CSV/XLSX con validación previa, control de cupo y progreso en tiempo real registro a registro.	`gestionar-empresa/includes/ajax-import.php`
Test de diagnóstico	Captura de respuestas, cálculo de resultados y asignación automática de cursos derivada del resultado.	`themes/astra-child/app/test`
Formación (LMS)	Cursos, lecciones, progreso, certificados y logros sobre LearnDash.	`plugins/sfwd-lms`
Venta directa	Catálogo de tests y cursos como productos WooCommerce con pasarelas Stripe y PayPal y embudos CartFlows.	`plugins/woocommerce`
Inicio de sesión único	SSO bidireccional con los otros dos sitios del ecosistema mediante token firmado.	`plugins/mi-sso-emisor`

Capacidad	Descripción	Implementación principal
Autoservicio de perfil	El empleado edita sus propios datos y contraseña, con replicación a los sitios hermanos.	`plugins/edit-subscriber-profile`
APIs de integración	Endpoints REST con token Bearer para consultar usuarios, crear empresas y actualizar suscriptores desde fuera.	`plugins/user-data-api`
Asistente conversacional	Widget de chat con IA conectado a un runtime externo, con token JWT de corta vida emitido por el propio sitio.	`plugins/efrenia-agents`
Herramientas internas	Suplantación de usuario, auditoría de vídeos, sustitución de audios, exportación de plugins, usuarios de prueba.	`varios plugins propios`
Endurecimiento HTTP	Cabeceras de seguridad, CSP en modo informe, captcha Turnstile en los formularios de acceso.	`mu-plugins/`

1.3 Tecnologías utilizadas

Tabla 2 — Pila tecnológica. Las versiones son las declaradas en las cabeceras de los archivos.

Capa	Componente	Versión / nota
Lenguaje	PHP	Mínimo declarado 7.4 en los plugins propios
CMS	WordPress	Mínimo declarado 5.8 (6.0 en un módulo). Versión real: pendiente de verificación
Base de datos	MySQL / MariaDB	InnoDB, `utf8mb4_unicode_520_ci`
LMS	LearnDash (`sfwd-lms`) + Course Grid, Certificate Builder, Achievements, Notifications, Elementor integration, WooCommerce integration	Suite completa
Comercio	WooCommerce + Stripe + PayPal Payments + CartFlows + Checkout Manager + exportación de pedidos	—
Identidad	Ultimate Member (registro y acceso en el frontal) + SSO propio	—
Evaluaciones	Watu Quiz 3.4.4 y Watu PRO 6.7.9	Coexisten con el test propio del tema
Presentación	Astra + tema hijo propio + Elementor y Elementor Pro	Tema hijo: 126 archivos, 10.552 líneas PHP
Frontend	jQuery, SweetAlert2 11, SheetJS 0.18.5 (desde CDN), CSS propio con variantes minificadas	Sin empaquetador ni paso de compilación
Entrega de medios	bunny.net (CDN y streaming), Vimeo, YouTube, Spotify	—

Capa	Componente	Versión / nota
Operación	Wordfence, SG Cachepress (SiteGround), All-in-One WP Migration, WP Mail SMTP, Yoast SEO, Loco Translate, Redirection	—
Idioma	Español (`es_ES` y `es_CO`), 48 MB de traducciones	—

> **Dato relevante sobre el origen del código**
>
> El tema hijo no fue escrito para Inspira. Su propia cabecera lo identifica como *«Tema hijo Astra para version 2.0 sistemadi»*, con autor *Global Authenticity LLC* y URI `sistemadi.com` (`themes/astra-child/style.css:1-11`). Dentro conviven tres tipos de test —`personal`, `pareja` y `padres`— y decenas de referencias a `aprobandome.com`, que son el producto anterior. **Inferencia:** el tema es código heredado y adaptado, lo que explica buena parte de la deuda técnica descrita en la sección 10.

2 Arquitectura del sistema

Inspira no es un sistema aislado: es uno de los tres vértices de un ecosistema que comparte identidad, cuentas y servicios auxiliares. Entenderlo exige mirar primero hacia fuera y después hacia dentro.

2.1 El ecosistema: tres sitios acoplados

El código referencia de forma explícita y repetida tres dominios hermanos, declarados como única fuente de verdad en `plugins/mi-sso-emisor/mi-sso-emisor.php:29-54`: `wmslatam.com` (programa «Liderazgo»), `octagono.wmslatam.com` («El Octágono») e `inspira.wmslatam.com` (este sitio). El plugin de SSO está diseñado para instalarse *idéntico* en los tres y autoidentificarse comparando `home_url()` con esa lista.

El acoplamiento va más allá del inicio de sesión: cuando Inspira crea un usuario, lo propaga a los otros dos sitios mediante llamadas REST servidor a servidor, y mantiene en `wp_usermeta` dos banderas —`wms` y `octagono`— que reflejan si esa persona tiene cuenta y programa activo en cada uno.

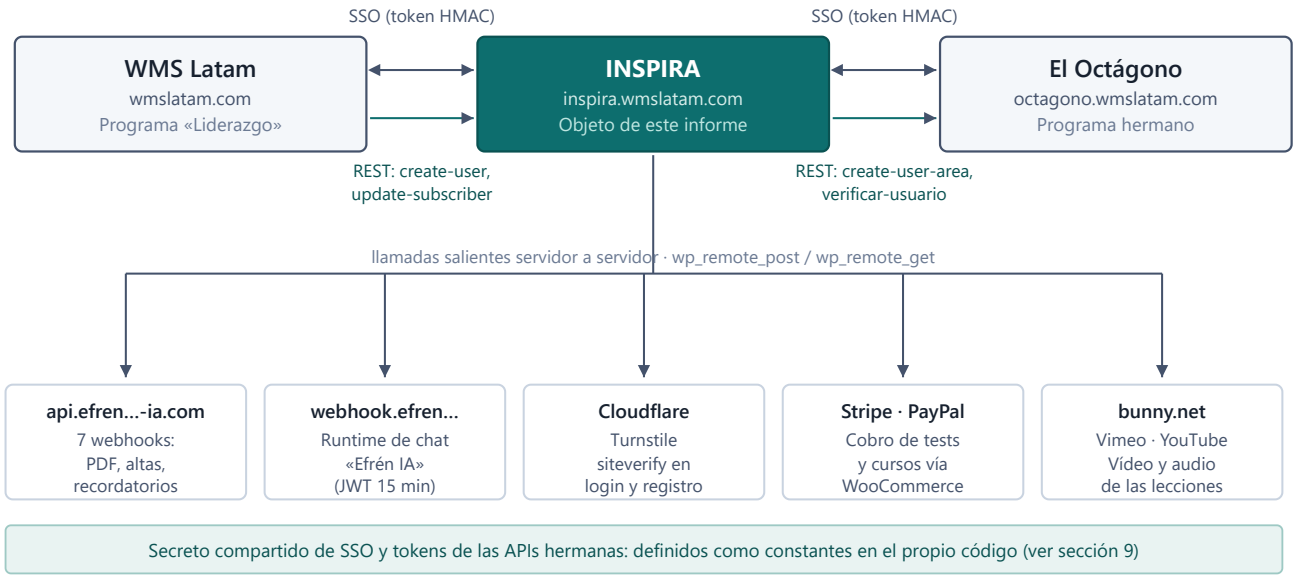
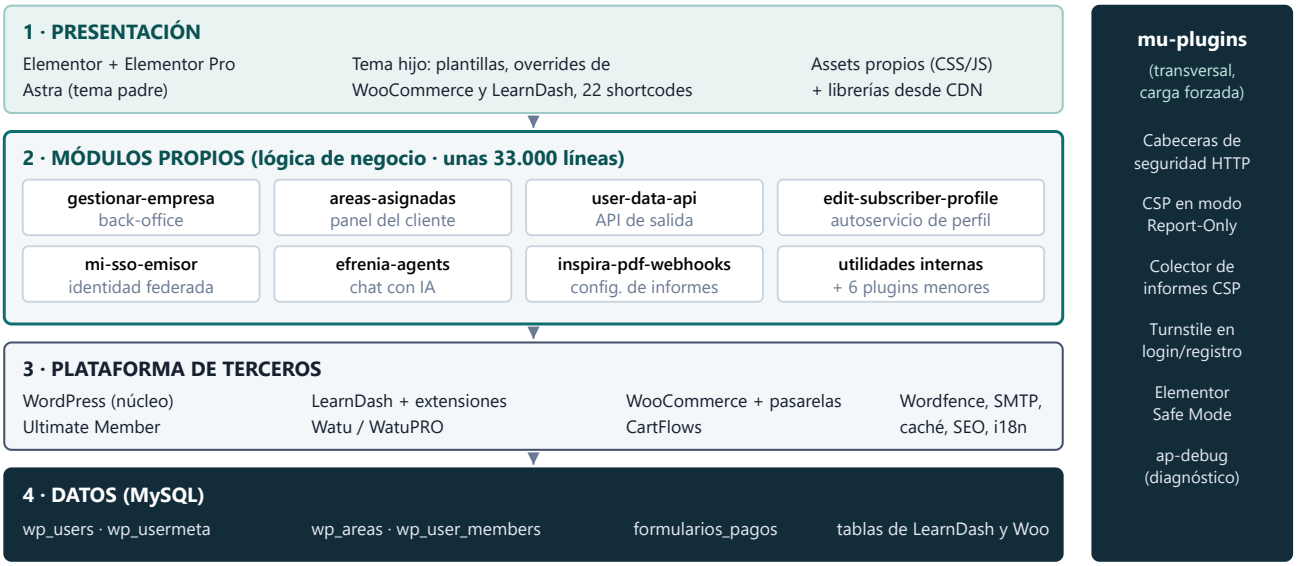


Figura 1 — Ecosistema WMS Latam. Los tres sitios se reconocen entre sí y propagan cuentas; los servicios externos cuelgan de Inspira mediante llamadas salientes. *Fuentes: mi-sso-emisor.php:29-54; areas-asignadas/includes/class-ajax.php:498-506; gestionar-empresa/includes/ajax-sync.php:193-406.*

2.2 Organización interna

Hacia dentro, la instalación se ordena en cuatro capas de aplicación más una capa transversal. Esa estratificación no está formalizada en el código, pero emerge con claridad de la distribución por carpetas y de las dependencias observadas.



Las flechas indican dependencia: cada capa consume la inferior. No hay inversión de dependencias, contenedor de servicios ni capa de acceso a datos compartida.

Figura 2 — Capas internas de la instalación. Los módulos propios no se comunican entre sí por interfaces: comparten estado a través de la base de datos y de `wp_usermeta`. *Inferencia a partir de la distribución de carpetas y de las dependencias observadas en el código.*

2.3 Relaciones entre componentes

Tres observaciones estructurales, todas comprobadas en el código:

- **El acoplamiento es por datos, no por interfaces.** `areas-asignadas` lee las tablas `wp_areas` y `wp_user_members` que crea y escribe `gestionar-empresa`, y ambos consultan el progreso de LearnDash por su cuenta. No existe una capa de servicio compartida ni contratos explícitos: si cambia una columna, hay que localizar a mano todos los consumidores.
- **Coexisten cuatro rutas de alta de usuario.** El back-office (`gestionar-empresa`), el panel del cliente (`areas-asignadas/includes/class-ajax.php:517`), la API `inspira-create-user-area` y los endpoints `aprobandome/v1` del tema. Cada una reimplementa su propia lógica de cupo, propagación y notificación. **Inferencia:** duplicación por evolución histórica, no por diseño.
- **El tema contiene lógica de negocio.** No solo presentación: las plantillas sobrescritas de WooCommerce ejecutan inserciones en base de datos y envíos de correo (`themes/astra-child/woocommerce/checkout/thankyou.php:130-148`), y los 15 manejadores AJAX del tema guardan resultados del test y asignan cursos.

3 Estructura de archivos y carpetas

El árbol analizado corresponde a `wp-content`. De los 591 MB totales, la gran mayoría es código de terceros y traducciones; el desarrollo propio ocupa menos de 10 MB pero concentra todo el valor funcional.

3.1 Raíz del proyecto

Tabla 3 — Contenido de primer nivel y responsabilidad

Ruta	Tamaño	Responsabilidad
`plugins/`	497 MB	61 plugins. 18 son desarrollo propio; el resto, terceros comerciales y libres.
`themes/`	47 MB	Astra, el tema hijo propio y cuatro temas por defecto de WordPress sin uso.
`languages/`	48 MB	Traducciones `es_ES` y `es_CO` de núcleo, plugins y temas, más archivos de Loco Translate.
`mu-plugins/`	56 KB	Cinco módulos de carga obligatoria: seguridad HTTP, CSP, Turnstile, Safe Mode de Elementor y depuración.
`novamira-sandbox/`	<1 KB	Directorio aislado con `.htaccess` y `web.config` que deniegan todo salvo CSS, JS y un índice vacío. Área de trabajo del plugin Novamira.
`upgrade/`	1 KB	Carpeta temporal de WordPress. Vacía.
`yaymail-logs/`	2 KB	Dos registros del plugin de plantillas de correo (fechas 2025-06-30 y 2026-03-17).
`index.php`	28 B	Silenciador estándar de listado de directorios.

> **Ausencia de control de versiones**
>
> No existe directorio `.git` ni ningún otro metadato de control de versiones en el material entregado. **Consecuencia directa sobre este análisis:** no es posible datar cambios, atribuir autoría real, distinguir código vivo de código abandonado por su última modificación, ni reconstruir por qué se tomó una decisión. Buena parte de lo que este informe clasifica como «inferencia» sería un hecho comprobable con historial.

3.2 Desarrollo propio: volumen y forma

Tabla 4 — Unidades de desarrollo propio, ordenadas por volumen de código PHP

Unidad	Versión	Líneas PHP	Forma del código
`themes/astra-child`	1.0.0	10.552	126 archivos. Funciones sueltas organizadas por carpetas temáticas (`app/`, `components/`, `inc/api/`, `hooks/`). Sin clases salvo `AppConstants` y `UserQueries`.
`plugins/edit-subscriber-profile`	2.9.0	4.130	Arquitectura en capas explícita: `domain/`, `http/`, `presentation/`, `support/`, `data/`. Una clase por responsabilidad. Es el módulo mejor estructurado del proyecto.

Unidad	Versión	Líneas PHP	Forma del código
`plugins/user-data-api`	1.16.0	3.887	11 clases con prefijo `UDA_`: cargador, autenticación, API, dominio (usuario, empresa, área), interpretación del test y administración.
`plugins/areas-asignadas`	2.5.0	3.824	Clases con prefijo `AP_` (loader, admin, ajax, database, frontend, helpers, security, settings, reminders) y plantillas parciales en `templates/parts/`.
`plugins/gestionar-empresa`	1.38.0	3.369	Funciones con prefijo `ge_` agrupadas por dominio AJAX (`ajax-areas`, `ajax-empresas`, `ajax-import`, `ajax-sync`, `ajax-usermeta`…). Sin clases.
`plugins/plugin-zip-exporter`	1.1.0	1.859	Namespaces y subcarpetas `Core/`, con activador, desactivador y limpieza programada.
`plugins/ld-audios-lecciones`	1.0.0	1.802	12 archivos, prefijo `LDAL_`.
`plugins/learndash-video-auditor`	2.8.2	1.421	6 archivos, prefijo `LDVA_`.
`plugins/efrenia-agents`	1.19.0	1.361	Dos archivos (`efrenia-agents.php` y `admin.php`) con funciones `efrenia_*`, incluida una implementación propia de firma JWT.
`mu-plugins/`	—	1.280	Cinco archivos independientes, muy comentados, con interruptores de emergencia por constante.
`plugins/mi-sso-emisor`	3.7.1	1.182	Archivo único por diseño: se instala idéntico en los tres sitios.
`plugins/usuario-testing`	1.0.0	865	4 archivos, prefijo `UT_`.
`Otros 6 plugins propios`	—	≈1.900	`inspira-pdf-webhooks`, `plugin-api-usuarios-inspira`, `wms-update-subscriber-api`, `inspira-create-user-area`, `sync-logout`, `login-como-usuario`, `blogs-comentarios`.

3.3 Anatomía del tema hijo

El tema concentra la mayor parte del código propio y merece desglose, porque su organización es la menos convencional del proyecto.

Tabla 5 — Carpetas del tema hijo y qué contienen

Ruta	Contenido y responsabilidad
`functions.php`	619 líneas. Punto de entrada: carga el resto, registra hooks y, al final, siembra en `wp_options` los tokens de las APIs del tema (ver sección 9).
`app/constants.php`	Clase `AppConstants`: identificadores numéricos fijos de productos, cursos y páginas de resultados. Punto de acoplamiento crítico con los datos de producción.
`app/*/ajax/`	15 manejadores AJAX: autenticación, test, resultados, carrito, cursos, parejas, menú, correos. Toda la interacción dinámica del frontal.
`app/queries/user-queries.php`	Clase `UserQueries`: acceso directo a la tabla `formularios_pagos`.

Ruta	Contenido y responsabilidad
`app/emails/`	Cuatro correos transaccionales (bienvenida, resultados, curso, pareja) separados en función y plantilla HTML.
`inc/api/post/`	Cinco endpoints REST propios del espacio `aprobandome/v1`: crear usuario administrador, crear usuario miembro, crear área, actualizar área, buscar id por correo.
`components/shortcodes/`	Shortcodes del frontal con su CSS adyacente (progreso del usuario, próximos cursos, detalles de membresía, validación de código de pareja…).
`woocommerce/`	Plantillas sobrescritas de checkout, gracias, pedido y cuenta. Contienen lógica de negocio además de marcado.
`learndash/ld30/lesson.php`	Única plantilla de LearnDash sobrescrita.
`ultimate-member/`	Plantillas de restablecimiento de contraseña y correos de Ultimate Member.
`assets/`	21 CSS y 18 JS. Cada CSS existe en versión fuente y `.min`, generadas a mano: no hay proceso de compilación.

Copia obsoleta en el árbol de plugins

`plugins/gestionar-empresa-1.31.0/` es una copia de seguridad de una versión anterior del plugin, mientras la activa declara 1.38.0. Además, sus archivos tienen los nombres corrompidos: la ruta completa quedó embebida en el nombre del fichero usando el carácter `U+F05C` en lugar de un separador de directorio —residuo de una descompresión defectuosa en Windows—. Es código muerto que confunde al navegar el proyecto y no debe permanecer en `wp-content`.

4 Funcionalidades y módulos

Esta sección describe cada módulo propio: qué resuelve, cómo lo hace y de qué depende. Los plugins de terceros se resumen al final por su papel en la arquitectura, no uno a uno.

4.1 Mapa de módulos propios

Tabla 6 — Módulos propios: responsabilidad, superficie y dependencias

Módulo	Responsabilidad	Superficie expuesta	Dependencias
`gestionar-empresa` *v1.38.0*	Back-office completo: empresas, áreas con cupo, suscriptores, importación masiva, editor de `wp_usermeta`, sincronización con los sitios hermanos y suplantación de usuario.	Menú de administración `gestionar-empresa` + submenú de configuración. 26 acciones AJAX con prefijo `ge_`.	Crea `wp_areas`. Escribe `wp_user_members`, `wp_users`, `wp_usermeta`. Consume APIs de WMS y Octágono. SweetAlert2 y SheetJS desde CDN.
`areas-asignadas` *v2.5.0*	Panel que ve la empresa cliente. Lista sus áreas y empleados con estado del test y porcentaje de avance; genera informes PDF (individual, por área y por empresa), exporta a Excel y envía recordatorios.	Shortcode `[areas_asignadas]`, página de ajustes y 9 acciones AJAX con prefijo `ap_`.	Lee `wp_areas`, `wp_user_members` y el progreso de LearnDash. Llama a los webhooks de PDF y a las APIs de alta de WMS y Octágono. Escritor XLSX propio (`class-xlsx-writer.php`).
`user-data-api` *v1.16.0*	Expone hacia fuera la ficha completa de un usuario (perfil, empresa, área, cursos de LearnDash y resultados del test interpretados) y permite crear empresas con sus áreas.	REST: `user-data-api/v1/user`, `/lookup` y `/company`. Página de administración con documentación y pruebas.	Token Bearer guardado en `wp_options`. LearnDash para los cursos. Clase propia de interpretación del test.
`edit-subscriber-profile` *v2.9.0*	Autoservicio: el empleado edita nombre, apellidos, edad, género, país, teléfono, cargo y contraseña. El correo se muestra pero no se edita. Replica los cambios a los sitios hermanos.	Shortcode `[edit_subscriber_profile]`, vista de cuenta de empresa, página de ajustes y manejador AJAX.	`wp_usermeta`. Endpoints `update-subscriber` de WMS y Octágono (con respaldo secuencial si falla el envío en paralelo).
`mi-sso-emisor` *v3.7.1*	Identidad federada entre los tres sitios: emite y verifica tokens, decide si mostrar el acceso a un sitio hermano según si el usuario tiene cuenta allí, y cachea esa respuesta.	Rutas reescritas `/sso-to-*`, shortcodes `[mi_sso_to_*]` y `[mi_sso_debug]`, REST `mi-sso/v1/check-user` y `/purge-cache`.	Secreto compartido en constante. Transitorios de WordPress para la caché. Los otros dos sitios del ecosistema.
`efrenia-agents` *v1.19.0*	Widget de chat con asistentes de IA. Sustituye a snippets anteriores. Decide quién ve cada asistente y emite un JWT de 15 minutos con la identidad y permisos del visitante.	Shortcodes `[efrenia_agent]` y `[asistente_virtual_chat]`, REST `efrenia/v1/token`, página de administración con comprobación de salud del runtime.	Runtime externo de agentes. Secreto JWT en opciones. Rol del usuario y existencia de resultados del test.

Módulo	Responsabilidad	Superficie expuesta	Dependencias
`inspira-pdf-webhooks` *v1.0.0*	Centraliza las URL de los webhooks que generan los informes PDF, que antes estaban escritas a mano en el tema y en `areas-asignadas`. Incluye envío de prueba contra datos reales.	Página de administración y API interna de consulta de URL.	`wp_options`. Servicio externo de generación de PDF.
`learndash-video-auditor` *v2.8.2*	Audita enlaces y códigos embebidos de YouTube y Vimeo en las lecciones, y reescribe los dominios de los reproductores (lo que la CSP de `mu-plugins` da por hecho).	Submenú bajo LearnDash.	Contenido de `wp_posts` de LearnDash.
`ld-audios-lecciones` *v1.0.0*	Detecta audios insertados en las lecciones (bloques, etiquetas `<audio>` y shortcodes) y permite sustituir sus URL sin tocar el resto del contenido.	Menú propio de administración.	`wp_posts`.
`plugin-zip-exporter` *v1.1.0*	Exporta cualquier plugin instalado como ZIP instalable. Único módulo propio con tarea programada (limpieza horaria de temporales).	Menú propio. Requiere capacidad `install_plugins`.	Sistema de archivos y WP-Cron.
`usuario-testing` *v1.0.0*	Creación y gestión de usuarios de prueba con asignación a empresa y área.	Menú propio `usuario-testing`.	`wp_user_members`, `wp_areas`.
`login-como-usuario` *v2.0.0*	Segunda implementación de suplantación de usuario, independiente de la que ya incluye `gestionar-empresa`.	Pantalla de administración.	Cookies de autenticación de WordPress.
`sync-logout` *v1.3*	Propaga el cierre de sesión a los tres sitios del ecosistema.	REST `custom/v1/logout`.	Los otros dos sitios.
`blogs-comentarios` *v1.1.0*	Muestra comentarios recogidos por formularios de Elementor, agrupados por capítulo.	Shortcode `[blog_comentarios]` y AJAX público de paginación.	Tablas `wp_e_submissions` de Elementor Pro.
`APIs de un archivo`	`plugin-api-usuarios-inspira` (consulta el rol por correo), `wms-update-subscriber-api` (actualización parcial de suscriptor), `inspira-create-user-area` (crea usuario con membresía de área).	REST `inspira/v1/*` y espacio propio, siempre con token Bearer.	`wp_users`, `wp_usermeta`, `wp_user_members`.

4.2 Los mu-plugins: capa transversal de seguridad

Los cinco archivos de `mu-plugins/` son, con diferencia, el código mejor documentado del proyecto. Sus comentarios citan un *«Informe de Seguridad y Habilitación de Dominios (Global Authenticity LLC / Grupo Aval)»* y describen un despliegue por fases con interruptor de emergencia por constante en `wp-config.php`.

Tabla 7 — Módulos de carga obligatoria

Archivo	Función y estado
`wms-security-headers.php`	Publica `X-Content-Type-Options`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy` y `Permissions-Policy` en modo activo, y `Content-Security-Policy` en **modo Report-Only**. Mantiene un inventario comentado de los orígenes que el navegador carga. **HSTS no se emite todavía**, por ser la única cabecera difícil de revertir. Se implementa en PHP y no en `.htaccess` por el tipo de servidor.
`wms-csp-report-collector.php`	Endpoint REST público que recibe los informes de violación de CSP y una pantalla de administración (`wms-csp-reports`) para revisarlos. Su apertura al público es intencional: así funciona el mecanismo de informes del navegador.
`wms-turnstile.php`	Captcha de Cloudflare Turnstile en el acceso y el registro, tanto de Ultimate Member como de `wp-login.php` y la recuperación de contraseña. Tiene modo monitorización y modo bloqueo, lleva estadísticas en una opción y falla de forma permisiva —con mensaje explicativo— si la red del cliente bloquea el dominio de Cloudflare.
`elementor-safe-mode.php`	Modo seguro de Elementor. Generado por el propio plugin, no es desarrollo propio.
`ap-debug.php`	Volcado de diagnóstico de cursos, `usermeta` y actividad de LearnDash para un usuario, activable con el parámetro `?ap_debug=<id>`. Protegido por `manage_options` y con consultas preparadas; aun así es código de depuración residente en producción.

4.3 Terceros: su papel en la arquitectura

- **LearnDash** es el sistema de registro del aprendizaje: cursos, lecciones, progreso y la tabla `wp_learndash_user_activity`. El código propio nunca lo sustituye, pero sí lo consulta de tres maneras distintas para calcular avance (ver sección 10).
- **WooCommerce** es la puerta de entrada comercial. Los tests y los paquetes de cursos son productos, identificados por id numérico fijo en `AppConstants`.
- **Ultimate Member** gestiona acceso y registro en el frontal; el tema sobrescribe sus plantillas de contraseña y el `functions.php` desactiva su control de acceso en la portada.
- **Watu y WatuPRO** son un motor de cuestionarios de terceros que **coexiste** con el test propio implementado en el tema. **Pendiente de verificación:** cuál de los dos está realmente en uso en producción; hay CSS propio para Watu, lo que sugiere uso real o reciente.
- **Novamira** y **Easy MCP AI** son servidores que exponen WordPress a agentes de IA. El primero declara en su propia cabecera que da *«acceso completo a WordPress mediante ejecución de PHP y operaciones de sistema de archivos»* y que es *«solo para entornos de desarrollo y staging»*; incluye un archivo `includes/abilities/execute-php.php`. Se tratan en la sección 9.

5 Base de datos

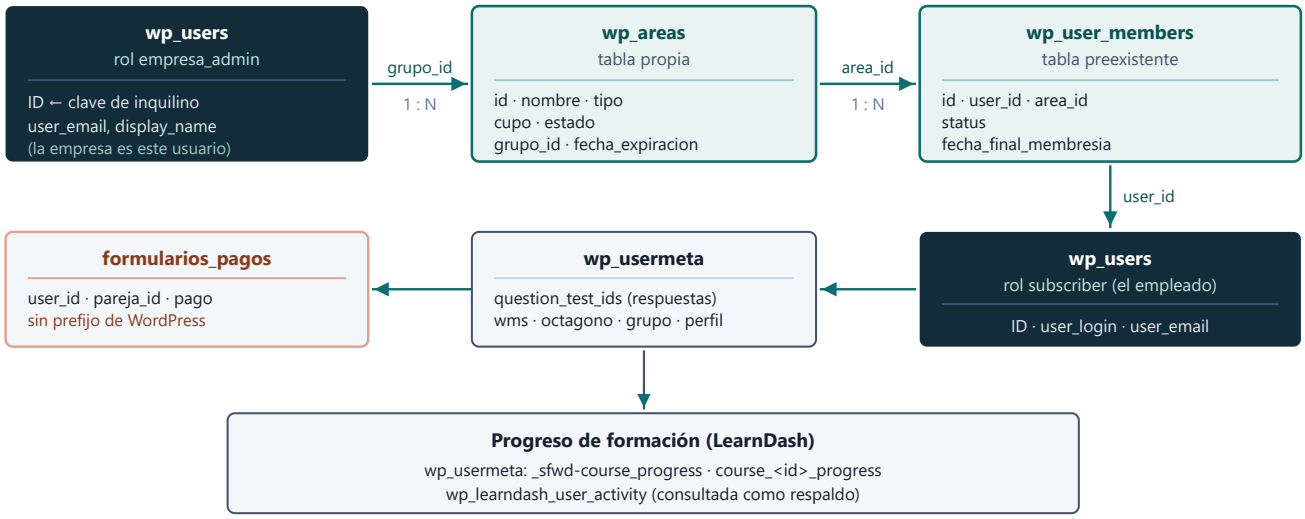
> **Límite de esta sección**
>
> No se dispuso de acceso a la base de datos. Todo lo que sigue se ha reconstruido leyendo las sentencias SQL, las llamadas a `dbDelta()` y los accesos a metadatos presentes en el código. Los esquemas reales —índices, tipos exactos, columnas que el código no toca— **están pendientes de verificación** contra la instalación.

5.1 Tablas personalizadas

Tabla 8 — Tablas no estándar utilizadas por el código propio

Tabla	Origen	Columnas observadas y papel
`wp_areas`	Creada por el código propio	Definición completa en `gestionar-empresa/includes/bootstrap.php:14-31`: `id`, `nombre`, `cupo`, `estado` (por defecto «Activo»), `tipo`, `grupo_id`, `fecha_expiracion`. InnoDB, `utf8mb4_unicode_520_ci`. **Clave del modelo multiempresa:** `grupo_id` contiene el `ID` del usuario `empresa_admin` propietario. No hay tabla de empresas: la empresa *es* el usuario administrador.
`wp_user_members`	Preexistente	Ningún módulo propio la crea, pero cuatro la leen y dos insertan en ella `(inspira-create-user-area.php:209; themes/astra-child/inc/api/post/user/create-user-member-post.php:138)`. Columnas observadas: `id`, `user_id`, `area_id`, `status`, `fecha_final_membresia`. Relaciona empleado con área. **Pendiente de verificación:** qué componente la creó originalmente y si tiene índices sobre `area_id` y `user_id`; de ello depende el rendimiento de todo el panel del cliente.
`formularios_pagos`	Preexistente (heredada)	Usada solo por el tema. Columnas observadas: `ID`, `user_id`, `pareja_id`, `resultados_id`, `pago`, `fecha_pago`, `tipo_resultado`, `coupon`, `correo_enviado`. Registra la compra de un test. **Se referencia sin el prefijo de WordPress** —nombre literal `formularios_pagos`— lo que impide instalar el sitio con otro prefijo o conviviendo con otra instalación en la misma base de datos.
`wp_e_submissions` `wp_e_submissions_values`	Elementor Pro	Consultadas en modo lectura por `blogs-comentarios` para mostrar los comentarios enviados por formularios.
`wp_learndash_user_activity`	LearnDash	Consultada directamente como último recurso para determinar si un curso está completado `(areas-asignadas/includes/class-database.php:345-355)`.

5.2 Modelo relacional



Todas las relaciones son lógicas: no se observan claves foráneas declaradas. La integridad referencial depende por completo del código de aplicación.

Figura 3 — Modelo de datos multiempresa. El aislamiento entre clientes se sostiene sobre una sola columna, `wp_areas.grupo_id`, comparada contra el identificador del usuario autenticado. *Fuentes: gestionar-empresa/includes/bootstrap.php:14-31; areas-asignadas/includes/class-database.php:27-130.*

5.3 Metadatos de usuario relevantes

El proyecto usa `wp_usermeta` como almacén de propósito general, incluidos datos que normalmente tendrían tabla propia.

Tabla 9 — Claves de `wp_usermeta` propias del proyecto

Clave	Escrita por	Contenido y observaciones
`question_test_ids`	Tema (AJAX del test)	**El dato más sensible del sistema.** JSON con todas las respuestas del test de diagnóstico, guardado como un solo valor de texto. De él derivan los resultados, la asignación de cursos y los informes PDF. Es la clave más referenciada del proyecto (15 apariciones).
`question_test_ids_temp`	Tema	Borrador de respuestas antes de consolidar.
`wms`	Sincronización	Bandera de cuenta y programa activo en `wmslatam.com`.
`octagono`	Sincronización	Bandera equivalente para `octagono.wmslatam.com`.
`grupo`	gestionar-empresa	Vínculo del usuario con su empresa, redundante con `wp_user_members → wp_areas.grupo_id`.
`nombre_cargado`	Importación	Nombre tal como venía en el CSV/XLSX, para cotejar altas masivas.
`membresia_activa`	Propia	Estado de la membresía.
`first_name, last_name, user_age, user_gender, country, phone_number, user_position, occupation`	Perfil	Datos personales del empleado, editables desde `edit-subscriber-profile` y replicados a los sitios hermanos. Conviven variantes antiguas (`user_phone_number`, `phone`).

Clave	Escrita por	Contenido y observaciones
`mi_autologin_token`, `mi_autologin_expires`	API del tema	Token de acceso automático emitido al crear un usuario por API.
`reset_password_token`, `reset_password_expire`	Tema	Restablecimiento de contraseña propio, paralelo al de WordPress.
`couple_one_id`, `second_couple_pending_test`	Tema (heredado)	Lógica de test «en pareja» del producto anterior. **Inferencia:** código muerto en Inspira.
`migrated`, `migrated_source`, `migrated_external_id`, `external_procurement_origin`	Migración	Trazas de una importación de usuarios desde un sistema anterior.
`_sfwd-course_progress`, `course_<id>_progress`	LearnDash / propio	Progreso por curso. La primera es de LearnDash; la segunda es un formato propio anterior que el código aún lee como respaldo.

5.4 Operaciones principales de lectura y escritura

Tabla 10 — Operaciones de datos de mayor impacto

Tipo	Operación	Origen
Lectura intensiva	Listado de empleados de un área con estado del test: `JOIN` de `wp_user_members` y `wp_users` con subconsulta `EXISTS` sobre `wp_usermeta`. Paginada a 10 registros.	`areas-asignadas/includes/class-database.php:131`
Lectura intensiva	Exportación a Excel: el mismo `JOIN` **sin paginar**, más el cálculo de avance curso a curso para cada usuario.	`areas-asignadas/includes/class-database.php:376`
Lectura	Verificación de pertenencia al inquilino antes de cada operación: `area_pertenece_a_grupo()` y `usuario_pertenece_a_grupo()`.	`class-database.php:45 y :60`
Escritura	Alta de empleado: `wp_insert_user()` + metadatos + `INSERT` en `wp_user_members` con fecha de fin de membresía heredada del área.	`class-database.php:108 · class-ajax.php:517`
Escritura	Guardado del test: `update_user_meta()` de `question_test_ids`, seguido de borrado y reasignación completa de los cursos del usuario.	`themes/astra-child/app/test/ajax/save-test.php:322`
Escritura	Registro de compra de test y marca de correo enviado en `formularios_pagos`, ejecutado desde una plantilla de WooCommerce.	`themes/astra-child/woocommerce/checkout/thankyou.php:130`
Escritura directa	Edición arbitraria de `wp_usermeta` (buscar, crear, editar, borrar cualquier clave de cualquier usuario) desde el back-office.	`gestionar-empresa/includes/ajax-usermeta.php`

Tipo	Operación	Origen
DDL en caliente	`SHOW TABLES LIKE` en ocho archivos distintos y comprobación en cada `admin_init` de si `wp_areas` existe, recreándola si no.	`gestionar-empresa/includes/bootstrap.php:34-41`

6 Flujos principales

Se documentan los tres recorridos que definen el sistema: el alta de un empleado por parte de su empresa, el ciclo test → resultados → cursos → informe, y el paso de un sitio a otro mediante SSO.

6.1 Alta de un empleado desde el panel de la empresa

Es el flujo con más efectos colaterales del proyecto: una sola pulsación crea una cuenta local y dispara cuatro llamadas salientes.

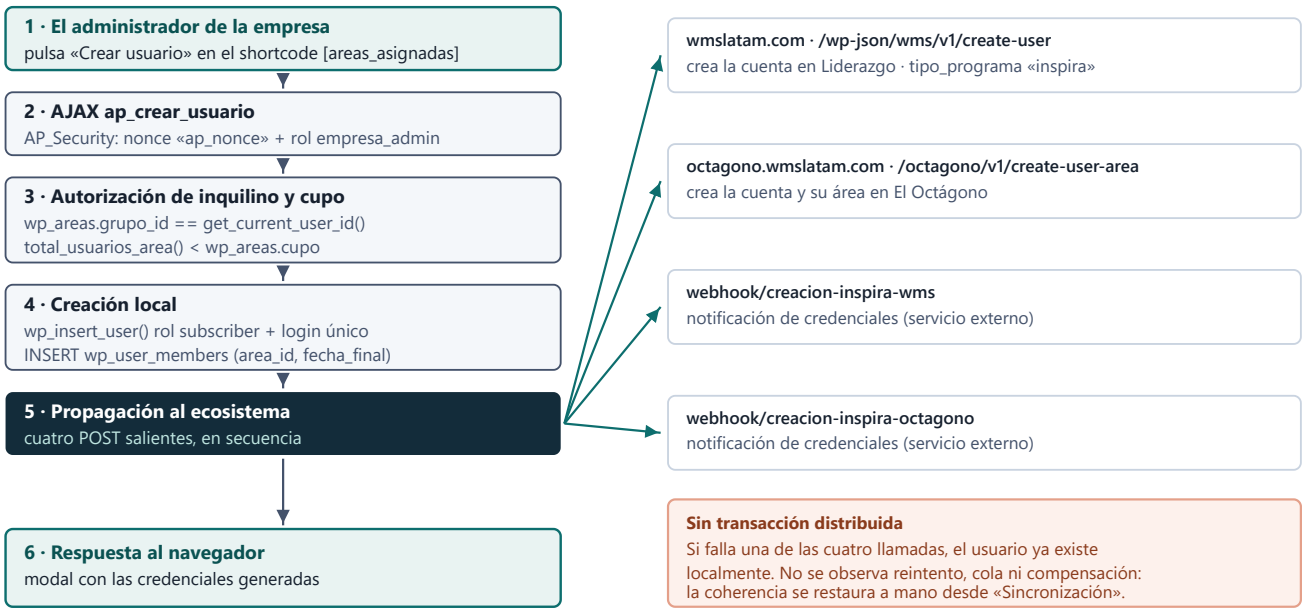
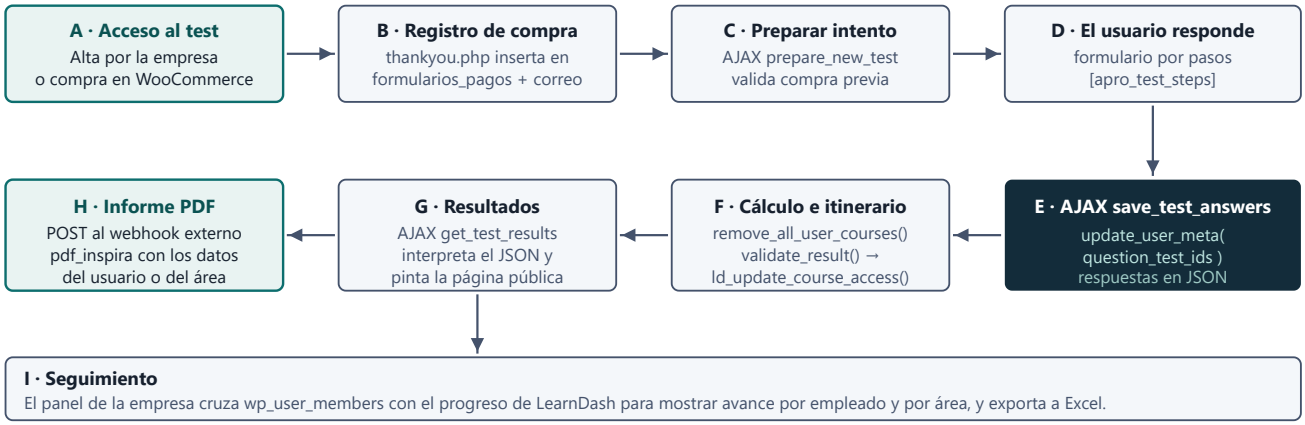


Figura 4 — Alta de usuario desde el panel de empresa. *Fuente: areas-asignadas/includes/class-ajax.php:517-790. El mismo patrón, con variantes, aparece en gestionar-empresa/includes/ajax-subscribers.php y en los endpoints REST de alta.*

6.2 Del test al itinerario de cursos y al informe

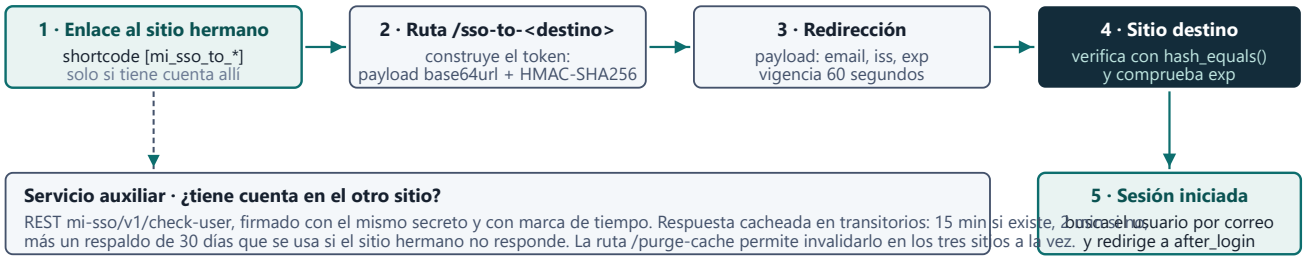
Este es el flujo que da sentido al producto. Reside casi íntegramente en el tema hijo.



Paso B: la escritura ocurre dentro de una plantilla de WooCommerce, no en un hook — se ejecuta al renderizar la página de agradecimiento.
Paso F: el itinerario se recalcula por completo en cada envío; los cursos previos se eliminan antes de reasignar.
Paso H: la maquetación del PDF es responsabilidad de un servicio externo; el sitio solo aporta los datos.

Figura 5 — Ciclo del test. *Fuentes: themes/astra-child/app/test/ajax/new-test.php y save-test.php; app/results/ajax/get-test-results.php; woocommerce/checkout/thankyou.php:130; areas-asignadas/includes/class-ajax.php:304-421.*

6.3 Inicio de sesión único entre sitios



El mismo archivo del plugin se instala en los tres sitios y se autoidentifica comparando home_url() con su lista de dominios; cada sitio es a la vez emisor y receptor.
Se aceptan alias heredados de la versión anterior del plugin («aprobandome» como emisor, rutas /sso-to-b) para no invalidar instalaciones antiguas.
El token no lleva identificador único de un solo uso: dentro de su ventana de 60 segundos es reutilizable (ver sección 9).

Figura 6 — Flujo de SSO. *Fuente: plugins/mi-sso-emisor/mi-sso-emisor.php:247-408 (emisión y verificación), :436-600 (comprobación remota y caché), :768-830 (purga).*

6.4 Otros recorridos relevantes

- **Importación masiva.** El back-office valida primero todo el archivo CSV/XLSX (`ge_import_validate`) y después crea los usuarios de uno en uno (`ge_import_create_one`), con progreso en tiempo real. El troceado es deliberado para no agotar el tiempo de ejecución de PHP. La lectura del archivo se hace en el navegador con SheetJS.
- **Edición de perfil con replicación.** El empleado guarda sus datos; `edit-subscriber-profile` actualiza `wp_usermeta` y envía los cambios a los endpoints `update-subscriber` de los dos sitios hermanos, con un camino de respaldo que reintenta uno por uno si el envío conjunto falla `(includes/http/class-esp-remote-sync.php:144-260)`.
- **Recordatorios.** El administrador de la empresa consulta quién no ha completado su registro o su test y dispara el aviso. **No hay tarea programada:** el envío es siempre manual, mediante AJAX, delegando el correo en un webhook externo `(areas-asignadas/includes/class-reminders.php:95)`.
- **Cierre de sesión federado.** Al cerrar sesión, `sync-logout` notifica a los otros dos sitios mediante `custom/v1/logout` para que invaliden también su cookie.

7 Integraciones externas

El sitio mantiene nueve integraciones salientes activas y expone once endpoints de entrada. Todas las salidas son servidor a servidor mediante la API HTTP de WordPress; no se observa ningún SDK de terceros instalado por los módulos propios.

7.1 Integraciones salientes

Tabla 11 — Servicios externos consumidos por el código propio

Destino	Propósito	Funcionamiento	Autenticación
`wmslatam.com /wp-json/wms/v1/create-user`	Alta de cuenta en el programa «Liderazgo»	`POST` con correo, contraseña y `tipo_programa: inspira` al crear un empleado. Si el usuario no se da de alta aquí, en WMS queda como usuario externo sin licencia.	Token Bearer **fijo en el código** (`class-ajax.php:499`)
`wmslatam.com /wp-json/wms/v1/update-subscriber`	Replicación de perfil	`POST` parcial: solo los campos modificados. Invocado desde la edición de perfil y desde la sincronización del back-office.	Token Bearer en opciones
`wmslatam.com /wp-json/wms-api/v1/verificar-usuario`	Comprobación de existencia de cuenta	Consulta previa a mostrar el acceso por SSO y a sincronizar. Resultado cacheado en transitorios.	Firma HMAC con marca de tiempo
`octagono.wmslatam.com /octagono/v1/create-user-area`	Alta de cuenta y área en El Octágono	`POST` equivalente al de WMS, con el área de destino.	Token Bearer **fijo en el código** (`class-ajax.php:506`)
`octagono.wmslatam.com /octagono/v1/update-subscriber /octagono-api/v1/verificar-usuario`	Replicación y comprobación	Simétricos a los de WMS.	Token Bearer / HMAC
`api.efrenmartinezortiz-ia.com`	Generación de informes PDF, notificación de altas y recordatorios	Siete webhooks distintos: `pdf_inspira` y `pdf_aprobandome` (maquetación del informe), `creacion-inspira`, `creacion-inspira-wms` y `creacion-inspira-octagono` (envío de credenciales), `credenciales_para_wms-inspira-array` (altas por lotes) y `recocrdatorio-inspira` (recordatorios; el nombre contiene una errata en el propio servicio). El sitio envía los datos y el servicio devuelve o entrega el documento.	URL del webhook como secreto; configurables desde `inspira-pdf-webhooks`
`webhook.efrenmartinezortiz-ia.com`	Runtime conversacional «Efrén IA»	El navegador habla con el runtime; el sitio solo emite un JWT de 15 minutos que acredita identidad, asistente autorizado y, en modo `results`, que el usuario tiene resultados. El plugin de administración consulta además un endpoint `/health`.	JWT firmado con secreto en opciones

Destino	Propósito	Funcionamiento	Autenticación
`challenges.cloudflare.com`	Protección anti-bot	Turnstile en los formularios de acceso, registro y recuperación. La validación del token se hace en el servidor contra `siteverify`. Falla de forma permisiva si el dominio está bloqueado en la red del cliente.	Clave secreta de sitio
`cdn.jsdelivr.net` `fonts.googleapis.com` `cdnjs.cloudflare.com`	Librerías y tipografías	SweetAlert2 11, SheetJS 0.18.5 y las fuentes Inter y Plus Jakarta Sans se cargan desde CDN público en las pantallas de administración, sin versión fijada en el caso de SweetAlert2 ni comprobación de integridad.	Ninguna
`bunny.net · Vimeo · YouTube · Spotify`	Medios de las lecciones	Reproductores embebidos en el contenido de LearnDash. `learndash-video-auditor` audita y reescribe esos enlaces, y la CSP de `mu-plugins` declara sus dominios.	Según el proveedor
`Stripe · PayPal`	Cobro	Mediante los plugins oficiales de WooCommerce. El código propio no toca el cobro: solo reacciona en la página de agradecimiento.	Gestionada por el plugin

7.2 Superficie de entrada

Tabla 12 — Endpoints REST expuestos por el código propio

Ruta	Función	Control de acceso observado
`user-data-api/v1/user` `user-data-api/v1/lookup`	Devuelve la ficha completa de un usuario por id o correo: perfil, empresa, área, cursos de LearnDash y resultados del test interpretados.	Token Bearer en opciones, comparado con `hash_equals()` `(class-uda-auth.php:66)`. Rechaza todo si no hay token configurado.
`user-data-api/v1/company`	Crea una empresa (usuario `empresa_admin`) con sus áreas, autogenerando campos no enviados.	Igual que el anterior.
`inspira/v1/update-subscriber`	Actualización parcial de un suscriptor. Si el usuario no tiene rol `subscriber`, solo procesa las banderas `wms` y `octagono` e informa de los campos ignorados.	Token Bearer **en constante del código**, con `hash_equals()` `(wms-update-subscriber-api.php:116)`.
`inspira/v1/create-user-area`	Crea un usuario de WordPress junto con su membresía de área.	Token Bearer en constante, con `hash_equals()` `(inspira-create-user-area.php:70)`.
`(espacio propio) consulta de rol por correo`	Devuelve el rol de un usuario de Inspira a partir de su correo.	Token Bearer en opciones, con `hash_equals()` `(plugin-api-usuarios-inspira/includes/class-auth.php:67)`.

Ruta	Función	Control de acceso observado

`aprobandome/v1/create-user`
`aprobandome/v1/create-user-member`
`aprobandome/v1/create-area`
`aprobandome/v1/update-area`
`aprobandome/v1/user-id-by-email`
	Cinco endpoints del **tema**: creación de usuarios administradores y miembros, gestión de áreas y búsqueda de identificador por correo.	Token Bearer leído de `wp_options`, con `hash_equals()`. **El valor por defecto de esas opciones está escrito en el código del tema** (ver sección 9).

`mi-sso/v1/check-user`	Responde si existe una cuenta para un correo dado, excluyendo roles configurados.	`permission_callback` abierto, con verificación interna de firma HMAC y marca de tiempo.
`mi-sso/v1/purge-cache`	Invalida la caché de existencia de cuenta en los tres sitios.	Igual que el anterior.
`custom/v1/logout`	Cierre de sesión federado.	`permission_callback` abierto.
`efrenia/v1/token`	Emite el JWT de 15 minutos para el widget de chat.	`permission_callback` abierto **por diseño**: la autorización real se hace dentro con `efrenia_can_see()`, que comprueba rol, interruptor del panel y existencia de resultados `(efrenia-agents.php:745-785)`.
`(colector de CSP)`	Recibe los informes de violación de la política de contenidos.	Abierto **por necesidad**: así funciona el mecanismo de informes del navegador.

Valoración de conjunto

Las APIs de entrada propias están, en general, **bien construidas**: usan `hash_equals()` para evitar ataques de medición de tiempo, rechazan las peticiones si no hay token configurado y documentan su propio contrato en pantallas de administración. Los tres `permission_callback` abiertos que no tienen una razón de diseño declarada son `custom/v1/logout` y las dos rutas de SSO, y estas últimas sí verifican firma internamente. El problema no está en el diseño de la autenticación, sino en **dónde viven los secretos** (sección 9).

8 Elementos propios de WordPress

Inventario de los mecanismos de extensión que usa el proyecto. Es la referencia práctica para quien tenga que modificarlo: indica dónde «engancha» cada cosa.

8.1 Shortcodes

Tabla 13 — Shortcodes registrados por el código propio

Etiqueta	Origen	Función
`[areas_asignadas]`	areas-asignadas	Panel completo de la empresa cliente. Es la pieza central del frontal B2B.
`[edit_subscriber_profile]`	edit-subscriber-profile	Formulario de autoservicio de perfil del empleado.
`[efrenia_agent]`	efrenia-agents	Widget de chat con IA, parametrizado por asistente, modo y altura.
`[asistente_virtual_chat]`	efrenia-agents	Variante del anterior.
`[mi_sso_to_*]`	mi-sso-emisor	Enlace de salto a un sitio hermano, con alias heredados. Solo se pinta si el usuario tiene cuenta allí.
`[mi_sso_debug]`	mi-sso-emisor	Diagnóstico del estado de SSO. **Código de depuración en producción.**
`[blog_comentarios]`	blogs-comentarios	Comentarios de formularios de Elementor agrupados por capítulo.
`[apro_test_steps]`	Tema	Formulario del test por pasos.
`[process_steps]`	Tema	Indicador de progreso del recorrido.
`[pending_test]`	Tema	Aviso de test pendiente.
`[personal_result_review]`	Tema	Revisión del resultado individual.
`[report-content-table]`	Tema	Índice del informe de resultados.
`[user-general-progress]`	Tema	Progreso general del usuario.
`[ids-cursos-learndash]`	Tema	Listado de cursos asignados.
`[view-user-membership-details]`	Tema	Detalles de la membresía.
`[group_detail]`	Tema	Detalle de grupo.
`[custom_forgot_password]` `[custom_reset_password]`	Tema	Recuperación de contraseña propia, en paralelo a la de WordPress y la de Ultimate Member.
`[sistemadi_login]` `[sistemadi_signup]`	Tema	Acceso y registro. **El nombre delata el producto de origen** del tema.

Etiqueta	Origen	Función
`[couple_pending_test]` `[validate_couple_code]` `[couples-report-content-table]`	Tema	Funcionalidad de test «en pareja». **Inferencia:** heredada y sin uso en Inspira.
`[bunnycdn_stream_video]`	bunny.net	De terceros, incluido por completitud del inventario.

8.2 Acciones AJAX

La interacción dinámica se construye sobre `admin-ajax.php`, no sobre la API REST. Se contabilizan **50 acciones** en el código propio.

Tabla 14 — Acciones AJAX por módulo y estado de su protección

Módulo	Acciones	Prefijo y ejemplos	Protección
`gestionar-empresa`	26	`ge_create_area`, `ge_import_validate`, `ge_list_usermeta`, `ge_switch_user`, `ge_sync_program_user`…	**CORRECTA** Nonce `ge_nonce` y `current_user_can('manage_options')` en los ocho archivos AJAX.
`areas-asignadas`	9	`ap_get_usuarios_area`, `ap_crear_usuario`, `ap_get_resultados_pdf`, `ap_exportar_usuarios_excel`…	**CORRECTA** `AP_Security::verify_ajax_request()`: nonce `ap_nonce` + rol, más comprobación de pertenencia al inquilino en cada operación.
`themes/astra-child`	15	`custom_login`, `custom_signup`, `save_test_answers`, `get_test_results`, `get_assign_course_ids_to_users`…	**DEFICIENTE** **Ninguna verifica nonce** y todas se registran además en `wp_ajax_nopriv_`. Detalle en la sección 9.
`Otros módulos propios`	≈6	Perfil, comentarios, usuarios de prueba, auditoría de vídeo.	Variable; `edit-subscriber-profile` valida mediante su clase `ESP_Request`.

8.3 Hooks y filtros

Tabla 15 — Puntos de enganche más significativos

Hook	Uso en el proyecto
`init`	Siembra de los tokens de las APIs del tema en `wp_options`; procesamiento de la suplantación de usuario y del retorno a la cuenta de administrador; registro de reescrituras de SSO.
`admin_init`	**Comprueba en cada carga del administrador si la tabla `wp_areas` existe** y la recrea si no. Cubre actualizaciones sin reactivar el plugin, a costa de una consulta `SHOW TABLES` por petición.
`rest_api_init`	Registro de los 21 endpoints REST propios.
`template_redirect`	Redirección del registro si ya hay sesión; desactivación del control de acceso de Ultimate Member en la portada; volcado de depuración de `ap-debug`.
`send_headers`	Emisión de las cabeceras de seguridad HTTP.

Hook	Uso en el proyecto
`admin_enqueue_scripts`	Carga de assets acotada por identificador de pantalla, con `wp_localize_script()` para pasar nonce y URL al JavaScript.
`admin_body_class`	Marca las pantallas del plugin para acotar los estilos de SweetAlert2 y **ocultar los avisos de otros plugins** mediante CSS en línea.

`um_after_login_fields`
`login_form`
`lostpassword_form`
	Inserción del captcha Turnstile en los formularios de Ultimate Member y de WordPress.

`wp_logout`	Limpieza de la cookie de suplantación y propagación del cierre de sesión a los sitios hermanos.
`woocommerce_*`	Enganches de comercio en `themes/astra-child/hooks/wc_hooks.php` y en la plantilla de agradecimiento.

8.4 Roles, capacidades y permisos

Tabla 16 — Modelo de autorización

Actor	Mecanismo	Alcance observado
Administrador del sitio	`manage_options`	Capacidad exigida en **62 puntos** del código propio. Es la única barrera de todo el back-office: no se definen capacidades propias, por lo que no es posible delegar «gestionar empresas» sin conceder el control total de la configuración del sitio.
Administrador de empresa	`empresa_admin`	Rol creado al activar `gestionar-empresa` clonando las capacidades de `subscriber` `(includes/helpers.php:68-73)`. La autorización **no se basa en capacidades sino en el nombre del rol** y en la comparación de `wp_areas.grupo_id` con el identificador del usuario autenticado.
Empleado	`subscriber`	Accede al test, a sus cursos de LearnDash, a sus resultados y a la edición de su propio perfil.
Usuario externo	`external_user`	Rol reconocido por el SSO como «sin acceso real» a WMS: oculta el enlace a ese sitio.
Otras capacidades	`install_plugins`, `list_users`, `edit_user`	Usadas puntualmente por el exportador de plugins y las herramientas de suplantación.

> **Dos definiciones del mismo rol**
>
> `areas-asignadas` permite configurar en sus ajustes qué rol es el de administrador de empresa `(class-settings.php:45)`, pero la comprobación efectiva usa una constante fija de la clase `AP_Security::ROLE_EMPRESA_ADMIN` `(class-security.php:13 y :19)`. Cambiar el ajuste no cambia quién tiene acceso. **Pendiente de verificación** sobre el resto de usos del ajuste, pero la ruta de autorización del shortcode y del AJAX ignora la configuración.

8.5 Tareas programadas

Solo existe **una** en todo el código propio: la limpieza horaria de archivos temporales del exportador de plugins (`plugin-zip-exporter/includes/Core/Activator.php:25`). Esto tiene una consecuencia funcional relevante: **no hay ningún proceso automático de negocio**. Los recordatorios a usuarios que no han completado su registro o su test, la sincronización con los sitios hermanos y la caducidad de las membresías por `fecha_expiracion` dependen de que una persona entre al panel y pulse un botón. **Pendiente de verificación:** si existen tareas configuradas fuera del código (cron del sistema o de terceros).

9 Seguridad

El nivel de seguridad del proyecto es muy desigual: los plugins propios recientes aplican buenas prácticas con rigor, mientras el tema hijo heredado concentra casi todos los defectos explotables. Esta sección separa lo uno de lo otro.

> **Alcance y método**
>
> Análisis estático, sin pruebas de penetración ni ejecución de código. No se verificó ninguna vulnerabilidad contra un entorno real: las descripciones indican por qué el código es explotable según su lectura, no un intento confirmado. Tampoco se evaluó la configuración del servidor, los permisos de archivo, `wp-config.php` ni las versiones instaladas de los plugins de terceros frente a sus avisos de seguridad conocidos. **Ningún secreto se reproduce en este documento:** se cita únicamente la ubicación donde está escrito.

9.1 Lo que está bien resuelto

Tabla 17 — Controles de seguridad correctamente implementados

Control	Evidencia
Aislamiento entre clientes	Toda operación del panel de empresa comprueba que el área o el usuario pertenecen al inquilino autenticado, con dos consultas dedicadas (`area_pertenece_a_grupo()`, `usuario_pertenece_a_grupo()`) invocadas en **cinco puntos distintos** del controlador AJAX. No se detectó ninguna acción de ese plugin que omita la comprobación `(areas-asignadas/includes/class-ajax.php:49, 214, 276, 314, 397)`.
Nonces y capacidades en el back-office	Los 26 puntos AJAX de `gestionar-empresa` y los 9 de `areas-asignadas` verifican nonce y autorización **sin excepciones**. En `areas-asignadas` está centralizado en una única clase declarada como «única fuente de verdad sobre quién puede hacer qué».
Comparación de tokens en tiempo constante	Las cinco APIs propias y el SSO usan `hash_equals()`, con el motivo documentado en el propio comentario del código `(user-data-api/includes/class-uda-auth.php:65-66)`.
Negación por defecto en las APIs	`user-data-api` rechaza **todas** las peticiones mientras no se configure un token, en lugar de aceptarlas con un valor vacío.
Suplantación de usuario	La implementación de `gestionar-empresa` exige `manage_options`, verifica nonce, firma la cookie de sesión de suplantación, emite cookie no persistente y la limpia al cerrar sesión `(includes/switch-user.php:40-371)`.
Consultas preparadas en los módulos nuevos	`areas-asignadas` y `gestionar-empresa` usan `$wpdb->prepare()` con marcadores y `esc_like()` en las búsquedas. Las únicas interpolaciones son nombres de tabla internos en `SHOW TABLES LIKE`, no entrada de usuario.
Endurecimiento HTTP	Cuatro cabeceras de seguridad activas, CSP en modo informe con colector propio, y captcha Turnstile en todos los formularios de acceso. Con despliegue por fases e interruptor de emergencia.
Ausencia de ejecución dinámica	No se encontró ni un solo uso de `eval()`, `exec()`, `shell_exec()`, `system()` ni `passthru()` en los 33.000 renglones de código propio.

9.2 Hallazgos

Se presentan ordenados por riesgo. La severidad valora el impacto sobre datos personales de clientes B2B y sobre los ingresos del producto.

Tabla 18 — Hallazgos de seguridad

#	Severidad	Hallazgo	Descripción y ubicación
H-1	CRÍTICA	Lectura de resultados de cualquier usuario sin autenticación	El manejador `get_test_results` toma `user_id` del cuerpo de la petición y lo usa tal cual si viene informado, recurriendo al usuario de la sesión solo si no viene. No comprueba nonce, no exige sesión —está registrado también en `wp_ajax_nopriv_`— ni verifica que ese identificador corresponda a quien pregunta. Devuelve nombre, apellidos y los resultados completos del test. `themes/astra-child/app/results/ajax/get-test-results.php:349-351 y :410`
H-2	CRÍTICA	Token de creación de administradores escrito en el código del tema	Un hook de `init` siembra cinco opciones de `wp_options` con un valor literal escrito en el propio archivo. Esas opciones son las credenciales de los endpoints `aprobandome/v1`, entre ellos el que **crea usuarios con rol de administrador de empresa**. Cualquiera que disponga de una copia del tema —un ZIP, un repositorio, un backup— posee la llave, y el valor solo se escribe si la opción no existe, por lo que rotarlo exige actuar en base de datos. El mismo valor está duplicado en dos archivos. `themes/astra-child/functions.php:594-618 · themes/astra-child/hooks/generate_secure_token_secret.php:5`
H-3	ALTA	Matriculación gratuita en cualquier curso	`get_assign_course_ids_to_users` acepta una lista de identificadores de curso del cliente y llama a `ld_update_course_access()` para cada uno. Exige sesión iniciada, pero **no comprueba compra, pertenencia ni nonce**, ni valida que los cursos formen parte del itinerario del usuario. Un empleado con sesión puede matricularse en todo el catálogo. `themes/astra-child/app/courses/ajax/assign-course-ids-to-users.php:30-64`
H-4	ALTA	Secretos de integración y de SSO en el código fuente	Cuatro secretos vivos están escritos como constantes o valores por defecto: los tokens de las APIs de alta de WMS y de Octágono `(areas-asignadas/includes/class-ajax.php:499 y :506; class-settings.php:48)`, el token de `inspira/v1/update-subscriber` `(wms-update-subscriber-api.php)`, el de `create-user-area` `(inspira-create-user-area.php)` y el **secreto compartido de SSO** `(mi-sso-emisor.php:17)`. Este último es el más grave de los cuatro: con él se puede forjar un token válido para suplantar a cualquier usuario por correo electrónico **en los tres sitios del ecosistema**, porque el plugin se instala idéntico en todos.
H-5	ALTA	Ausencia total de nonces en el frontal	Los 15 manejadores AJAX del tema no verifican nonce y todos están registrados en `wp_ajax_nopriv_`. Entre ellos: acceso, registro, cierre de sesión, añadir al carrito, guardar las respuestas del test y enviar correos. Esto permite falsificación de petición entre sitios (CSRF) en operaciones con efecto real. `themes/astra-child/app/*/ajax/*.php`

#	Severidad	Hallazgo	Descripción y ubicación
H-6	ALTA	Plugins de acceso total para agentes de IA	`novamira` (v1.12.13, 6,4 MB) declara en su cabecera dar *«acceso completo a WordPress mediante ejecución de PHP y operaciones de sistema de archivos»* y ser *«solo para entornos de desarrollo y staging»*; incluye `includes/abilities/execute-php.php`. Junto a él, `easy-mcp-ai` (v2.2.0, 12 MB, 243 herramientas). Si están activos en producción constituyen ejecución remota de código por diseño, y su superficie depende por completo de cómo estén autenticados. **Pendiente de verificación:** sin acceso a la base de datos no es posible saber si están activos. El directorio `novamira-sandbox/` tiene `.htaccess` y `web.config` restrictivos, lo que indica una instalación real y no un residuo.
H-7	MEDIA	Inyección SQL en consultas del tema	Tres consultas interpolan variables sin `prepare()` sobre la tabla `formularios_pagos`. Los identificadores numéricos llegan convertidos a entero en las rutas observadas, pero `$test_type` entra como cadena dentro de comillas simples. El riesgo real depende de la procedencia del parámetro en cada llamada, que no es uniforme. `themes/astra-child/app/queries/user-queries.php:8 y :13 · app/test/ajax/new-test.php:46`
H-8	MEDIA	Respuestas del test guardadas sin validar	`save_test_answers` escribe `$_POST['answers']` directamente en `wp_usermeta` sin sanear ni validar su estructura. Ese valor se decodifica después para calcular resultados y asignar cursos, y se envía a un servicio externo para maquetar el PDF. `themes/astra-child/app/test/ajax/save-test.php:322-323`
H-9	MEDIA	Token de SSO reutilizable dentro de su ventana	El token lleva emisor, correo y caducidad a 60 segundos, y se verifica con `hash_equals()`, pero **no incluye identificador de un solo uso** ni se registra su consumo. Quien lo capture —por ejemplo en el historial del navegador, un registro de proxy o una cabecera `Referer`— puede reutilizarlo durante ese minuto. `mi-sso-emisor.php:283-298 y :361-392`
H-10	MEDIA	Autorización basada en nombre de rol, no en capacidades	El rol `empresa_admin` se crea clonando las capacidades de `subscriber` y la autorización se decide comprobando si la cadena del rol está presente. No se define ninguna capacidad propia. Consecuencias: un administrador del sitio no puede delegar la gestión de empresas sin dar `manage_options`; un usuario con dos roles no se comporta de forma predecible; y el ajuste que permite cambiar el nombre del rol no tiene efecto sobre el control de acceso real (sección 8.4).
H-11	BAJA	Dos mecanismos de suplantación y herramientas de administración acumuladas	`login-como-usuario` duplica la funcionalidad de suplantación que ya tiene `gestionar-empresa`, con una implementación independiente. Junto a `code-snippets` (ejecución de PHP desde el administrador), `wp-file-manager`, `all-in-one-wp-migration` y `plugin-zip-exporter`, la superficie de ataque disponible para una cuenta de administrador comprometida es muy amplia. Cada herramienta es legítima por separado; el problema es la acumulación.

#	Severidad	Hallazgo	Descripción y ubicación
H-12	BAJA	Código de depuración en producción	`mu-plugins/ap-debug.php` expone un volcado de metadatos y actividad por parámetro de URL, y el shortcode `[mi_sso_debug]` muestra el estado del SSO. Ambos están protegidos —el primero por `manage_options` y con consultas preparadas— pero no deberían residir en un entorno productivo.
H-13	BAJA	Librerías de CDN sin versión fijada ni integridad	SweetAlert2 se carga como `sweetalert2@11`, un rango de versiones, sin atributo `integrity`, en pantallas donde se gestionan usuarios y contraseñas. Un compromiso del CDN afectaría al back-office. `gestionar-empresa/includes/bootstrap.php:88-115`

9.3 Tratamiento de información sensible

- **Datos personales y psicométricos.** El sistema almacena nombre, apellidos, edad, género, país, teléfono, cargo y las respuestas completas del test de diagnóstico de empleados de empresas cliente. En el marco de una normativa de protección de datos, las respuestas del test son el activo más sensible y son precisamente las que expone el hallazgo H-1.
- **Contraseñas generadas.** El alta de empleado genera la contraseña en el servidor y la devuelve en la respuesta AJAX para mostrarla en un modal, además de enviarla a dos sitios hermanos y a dos webhooks externos. **Pendiente de verificación:** qué hace cada destino con ese dato y si queda registrado en sus trazas.
- **Tokens de acceso automático.** Las APIs del tema emiten un `mi_autologin_token` de 40 caracteres con caducidad en `wp_usermeta`. Es un mecanismo de acceso sin contraseña cuya superficie depende de cómo se transporte.
- **Registros.** Se observan llamadas a `error_log()` en los módulos de Turnstile y de depuración. **Pendiente de verificación:** si algún registro recoge contraseñas o tokens; en el código revisado no se detectó ninguno que lo haga.

10 Rendimiento y mantenimiento

El sistema funciona, pero su coste de evolución crece con cada módulo añadido. Esta sección identifica los cuellos de botella medibles en el código y los factores que encarecen cualquier cambio futuro.

10.1 Cuellos de botella

Tabla 19 — Puntos de coste identificados en el código

Punto	Problema	Impacto esperado
Exportación a Excel del panel de empresa `class-database.php:376`	La consulta base recupera **todos** los empleados de las áreas seleccionadas sin paginar y, para cada uno, `get_cursos_y_avance_usuario()` recorre sus cursos invocando `learndash_course_progress()` una vez por curso. Esa función de LearnDash ejecuta varias consultas por llamada.	Crecimiento multiplicativo: usuarios × cursos. Para un área de 200 empleados con 10 cursos son más de 2.000 cálculos de progreso en una sola petición. Riesgo real de agotar el tiempo de ejecución de PHP.
Cálculo de progreso en cascada `class-database.php:300-356`	Cada curso se evalúa probando cuatro fuentes en orden hasta encontrar un valor: la función de LearnDash, la meta `_sfwd-course_progress`, una meta propia heredada `course_<id>_progress` y, por último, una consulta a `wp_learndash_user_activity`. En el peor caso se pagan las cuatro.	La cascada es defensiva y está bien comentada, pero cuadruplica el coste del caso desfavorable y perpetúa un formato de datos obsoleto.
Comprobación de tablas en cada petición `bootstrap.php:34-41`	Un hook de `admin_init` ejecuta `SHOW TABLES LIKE` en cada carga del administrador para recrear `wp_areas` si falta. El mismo patrón aparece en ocho archivos más para `wp_user_members`.	Coste bajo por petición pero constante y evitable. Debería resolverse con una opción de versión de esquema.
Lógica de negocio en plantilla `thankyou.php:130-148`	La inserción en `formularios_pagos` y el envío de correos ocurren al **renderizar** la página de agradecimiento, no en un hook del ciclo de vida del pedido. Hay una guarda por el campo pago, pero la operación se reevalúa en cada visita o recarga de esa página.	Correos duplicados y trabajo repetido ante recargas; imposibilidad de reprocesar el pedido sin que el cliente vuelva a abrir esa URL.
Respuestas del test en un único metadato	Todas las respuestas viven en un valor de texto JSON (`question_test_ids`). Cualquier estadística agregada — medias por área, comparativas, tendencias— obliga a leer y decodificar fila por fila en PHP.	Imposibilita agregar en SQL. El coste de cualquier informe nuevo es lineal en número de usuarios.
Ausencia de caché aplicativa	Solo el SSO usa transitorios. Los paneles recalculan el progreso en cada carga; no se observa uso de `wp_cache_*` ni de transitorios en `areas-asignadas`.	Todo el coste de la sección anterior se paga en cada visita al panel.
Llamadas externas sincrónicas	El alta de un usuario realiza cuatro `POST` salientes en secuencia dentro de la propia petición AJAX, y la generación de PDF espera la respuesta del servicio externo.	El tiempo de respuesta percibido es la suma de las latencias de terceros. Un servicio lento bloquea la interfaz del cliente.

Punto	Problema	Impacto esperado
Volumen de plugins	61 plugins, entre ellos dos suites de IA que suman 18 MB y 1.519 archivos PHP, y dos motores de cuestionarios (Watu y WatuPRO) junto al test propio.	Cada plugin activo añade coste de carga a toda petición. **Pendiente de verificación:** cuántos están realmente activos.

10.2 Duplicación de código y lógica

Tabla 20 — Lógica implementada más de una vez

Funcionalidad	Veces	Implementaciones
Alta de usuario con área	4	`gestionar-empresa/ajax-subscribers.php`, `areas-asignadas/class-ajax.php:517`, `inspira-create-user-area` y `themes/astra-child/inc/api/post/user/create-user-member-post.php`. Cada una repite control de cupo, generación de credenciales, propagación y notificación.
Suplantación de usuario	2	`gestionar-empresa/includes/switch-user.php` y el plugin `login-como-usuario`.
Actualización de suscriptor por API	2	`wms-update-subscriber-api` (plugin dedicado) y las rutas `aprobandome/v1` del tema.
Consulta de usuario por correo	3	`user-data-api/v1/lookup`, `plugin-api-usuarios-inspira` y `aprobandome/v1/user-id-by-email`.
Recuperación de contraseña	3	La de WordPress, la de Ultimate Member (con plantillas sobrescritas) y una propia en el tema con metadatos `reset_password_token`.
Motor de cuestionarios	2	Watu/WatuPRO y el test propio del tema, con CSS propio para ambos.
Vínculo usuario-empresa	2	La cadena `wp_user_members → wp_areas.grupo_id` y el metadato redundante `grupo`.
Progreso por curso	2	El formato de LearnDash y el formato propio heredado `course_<id>_progress`.

10.3 Factores que dificultan la evolución

- **Identificadores de producción incrustados en el código.** `AppConstants` fija por número los productos de los tests, los paquetes de cursos, cada curso individual y las páginas de resultados; `save-test.php` asigna además el curso `3045` de forma fija. **Consecuencia práctica:** el código no se puede desplegar en otro entorno —ni en un entorno de pruebas realista— sin reproducir exactamente esos identificadores en la base de datos. Esto es, por sí solo, el mayor obstáculo para montar un entorno de preproducción fiable.
- **Nombre de tabla sin prefijo.** `formularios_pagos` se referencia literalmente, lo que impide instalar el sitio con otro prefijo o compartir base de datos.
- **Código heredado indistinguible del vivo.** La funcionalidad de test «en pareja» y «de padres», los shortcodes `sistemadi_*` y las referencias a `aprobandome.com` conviven con el código activo sin ninguna marca que los separe. Sin historial de versiones no es posible decidir con seguridad qué se puede borrar.

- **Sin control de versiones.** No hay historial, ni atribución fiable, ni posibilidad de revertir. Las versiones solo constan como texto en las cabeceras de los plugins, y la autoría declarada es inconsistente: aparecen «Cristian», «Codex», «WMS Latam», «Aprobándome», «Global Authenticity», «Tu Empresa» y «Efrén IA».
- **Sin gestión de dependencias ni compilación.** No existe `composer.json` ni `package.json` en ningún módulo propio. Los CSS minificados se mantienen a mano junto a sus fuentes, con el riesgo de que divergan.
- **Sin pruebas automatizadas.** No se encontró ningún archivo de test, configuración de PHPUnit ni integración continua. Cualquier cambio en el cálculo de resultados o en la asignación de cursos solo se puede validar manualmente.
- **Dos convenciones de arquitectura.** `edit-subscriber-profile` y `user-data-api` siguen un diseño por capas con una clase por responsabilidad; `gestionar-empresa` y el tema son funciones globales con prefijo. Un desarrollador nuevo debe aprender ambos estilos.
- **Copia obsoleta en el árbol.** `gestionar-empresa-1.31.0/`, con nombres de archivo corrompidos, además de los cuatro temas por defecto de WordPress sin uso.

> **Lo que sí facilita el mantenimiento**
>
> Conviene no perder de vista el otro lado: los `mu-plugins` y los plugins propios recientes están **excepcionalmente bien comentados**, con explicaciones del «por qué» y no solo del «qué», advertencias explícitas sobre qué no cambiar y motivos documentados de cada decisión. `inspira-pdf-webhooks` nació precisamente para eliminar URL repetidas a mano, y `edit-subscriber-profile` demuestra que el equipo sabe estructurar código en capas. La capacidad técnica existe; lo que falta es aplicarla retroactivamente al tema.

11 Configuración y despliegue

> **Límite de esta sección**
>
> No se dispuso de `wp-config.php`, de la configuración del servidor web, de la base de datos ni de documentación de despliegue. Lo que sigue es la **reconstrucción** de los requisitos y los pasos a partir de las constantes y opciones que el código espera encontrar. No sustituye a un procedimiento de despliegue probado; sirve para redactarlo.

11.1 Requisitos del entorno

PHP	7.4 o superior (mínimo declarado en los plugins propios). Extensiones necesarias según el uso observado: `json`, `mbstring`, `zip` (exportador de plugins y escritor XLSX) y `openssl` (firmas HMAC y JWT).
WordPress	5.8 o superior; un módulo declara 6.0. **Pendiente de verificación:** versión real instalada.
Base de datos	MySQL o MariaDB con soporte de `utf8mb4_unicode_520_ci` e InnoDB.
Servidor web	Los comentarios de `wms-security-headers.php` indican que las cabeceras se emiten desde PHP **y no desde `.htaccess`** por el tipo de servidor de los dos dominios. El directorio `novamira-sandbox/` incluye a la vez `.htaccess` y `web.config`, por lo que la protección depende de cuál de los dos lea el servidor. **Pendiente de verificación.** La presencia de SG Cachepress apunta a alojamiento en SiteGround.
Correo saliente	WP Mail SMTP instalado: se requiere un proveedor SMTP configurado. El sistema envía credenciales, resultados y recordatorios.
Conectividad saliente	Imprescindible hacia `wmslatam.com`, `octagono.wmslatam.com`, `api.efrenmartinezortiz-ia.com`, `webhook.efrenmartinezortiz-ia.com` y `challenges.cloudflare.com`. Sin ella, el alta de usuarios, los informes PDF y los recordatorios fallan parcial y silenciosamente.
Herramientas de construcción	**Ninguna.** No hay `composer.json`, `package.json` ni paso de compilación: el código se despliega tal cual.

11.2 Configuración por constantes

Las siguientes constantes se leen de `wp-config.php` (o de un archivo equivalente). Se agrupan por módulo.

Tabla 21 — Constantes de configuración esperadas por el código

Constante	Efecto
`WMS_SECURITY_HEADERS_DISABLED`	Interruptor de emergencia: desactiva todas las cabeceras de seguridad sin necesidad de un despliegue.
`WMS_CSP_ENFORCE`	Pasa la política de contenidos de modo informe a modo bloqueo. **Fase 3 del plan documentado; aún no activada.**

`WMS_CSP_COLLECTOR_DISABLED`
`WMS_CSP_RATE_LIMIT`
`WMS_CSP_MAX_BODY`
`WMS_CSP_MAX_SIGNATURES`
	Control del colector de informes de CSP: desactivación, límite de peticiones, tamaño máximo del cuerpo y número de firmas distintas que se conservan.

Constante	Efecto
`WMS_TURNSTILE_SITE_KEY`	Clave pública del captcha. Sin ella, el widget no se pinta.
`WMS_TURNSTILE_ENFORCE`	Alterna entre modo monitorización (registra lo que habría bloqueado) y modo bloqueo real.
`WMS_TURNSTILE_DISABLED`	Interruptor de emergencia del captcha.
`EFRENIA_AGENTS_JWT_SECRET`	Secreto de firma del token del chat. Si falta, el endpoint responde error 500 y el asistente no funciona.
`EFRENIA_RUNTIME_URL`	URL base del runtime de agentes.
`WMS_INSPIRA_UPDATE_SUBSCRIBER_API_TOKEN` `INSPIRA_API_TOKEN`	Tokens Bearer de dos APIs de entrada. **Están definidos dentro del propio archivo del plugin**, no en `wp-config.php` (hallazgo H-4).
`MI_SSO_SECRET`	Secreto compartido del SSO, definido en el propio plugin y que debe ser idéntico en los tres sitios (hallazgo H-4).

11.3 Configuración por pantallas de administración

El resto de la configuración vive en `wp_options` y se edita desde las pantallas de cada módulo. Esto es un acierto de diseño —permite rotar tokens y cambiar webhooks sin tocar código—, pero implica que **un despliegue limpio no funciona hasta que alguien recorre esas pantallas**.

Tabla 22 — Opciones de configuración y dónde se editan

Opción	Pantalla	Contenido
`ap_ajustes`	Áreas Asignadas → Ajustes	Rol de administrador de empresa, tokens de WMS y Octágono, vista de solo lectura para el super-administrador.
`ap_webhook_recordar_registro` `ap_webhook_recordatorio_test`	Áreas Asignadas → Recordatorios	URL de los webhooks de aviso.
`ipw_webhooks`	Webhooks de PDF	URL de los generadores de informes, con envío de prueba contra datos reales.
`uda_bearer_token`	User Data API → API info usuario	Token de la API de consulta. Mientras esté vacío, la API rechaza todo.
`insu_bearer_token`	API Usuarios Inspira	Token de la API de consulta de rol.
`esp_plugin_settings` `esp_remote_endpoints`	Editar Perfil de Suscriptor	Campos editables y endpoints de replicación a los sitios hermanos.
`ge_settings_group`	Usuarios & Empresas → Configuración	Integraciones del back-office.
`efrenia_group`	Efrén IA	Asistentes, interruptor general, restricción a probadores y comprobación de salud del runtime.

Opción	Pantalla	Contenido
`aprobandome_create_admin_user` y 4 más	—	**Sin pantalla.** Se siembran automáticamente con un valor del código y solo se pueden cambiar en base de datos (hallazgo H-2).

11.4 Secuencia de instalación reconstruida

1. **Entorno.** WordPress sobre PHP 7.4+ y MySQL con `utf8mb4`. Prefijo de tablas `wp_`: **obligatorio**, porque `formularios_pagos` se referencia sin prefijo y el resto del código asume `wp_` en las rutas documentadas.
2. **Constantes.** Añadir a `wp-config.php` las de la tabla 21 que correspondan al entorno.
3. **Temas.** Instalar Astra y activar el tema hijo. Al activar, el hook de `init` siembra las cinco opciones de token de las APIs del tema.
4. **Plugins de terceros.** LearnDash y sus extensiones, WooCommerce con las pasarelas, Ultimate Member, Elementor Pro, y los de operación. **Pendiente de verificación:** el orden de activación importa, porque varios módulos propios comprueban la existencia de funciones de LearnDash.
5. **Plugins propios.** Activar `gestionar-empresa` **primero**: su rutina de activación crea el rol `empresa_admin` y la tabla `wp_areas`. El resto depende de ambos.
6. **Tabla pendiente.** `wp_user_members` no la crea ningún módulo propio. En una instalación nueva hay que crearla a mano o importarla. **Este es el principal punto ciego del despliegue** y debe resolverse antes de intentar reproducir el entorno.
7. **Datos de contenido.** Crear los cursos, los productos y las páginas de resultados con los identificadores numéricos exactos que espera `AppConstants`, o modificar esa clase. Sin esto, el itinerario de cursos y las páginas de resultados no funcionan.
8. **Configuración.** Recorrer las pantallas de la tabla 22 e introducir tokens y URL de webhooks.
9. **Ecosistema.** Instalar `mi-sso-emisor` en los tres sitios con el mismo secreto, y dar de alta en WMS y Octágono los tokens que Inspira usará para llamarlos.
10. **Verificación.** Usar los envíos de prueba que incorporan `inspira-pdf-webhooks` y `user-data-api`, y el `/health` del runtime de IA.

> **Conclusión operativa**
>
> El proyecto **no es reproducible** a partir del código. Reconstruirlo exige, además de los archivos, un volcado de base de datos con los identificadores de contenido correctos, los valores de las opciones, la tabla `wp_user_members` y las credenciales de las integraciones. En la práctica, el único procedimiento de despliegue viable hoy es clonar el sitio completo —para lo que está instalado All-in-One WP Migration—, lo que explica su presencia pero deja al proyecto sin un entorno de pruebas independiente.

12 Conclusiones y recomendaciones

12.1 Valoración de conjunto

Inspira WMS es un producto **funcionalmente completo y técnicamente desigual**. Resuelve un problema real —formación corporativa con diagnóstico previo, gestión delegada por empresa y medición del avance— y lo hace con una capa propia de 33.000 líneas que cubre todo el recorrido, desde la venta hasta el informe.

La calidad del código traza una línea muy nítida. Los módulos escritos recientemente para Inspira (`areas-asignadas`, `gestionar-empresa`, `user-data-api`, `edit-subscriber-profile` y los `mu-plugins`) aplican nonces, capacidades, consultas preparadas, comparación de tokens en tiempo constante y aislamiento entre clientes de forma sistemática, y están comentados con un cuidado poco habitual. El **tema hijo heredado** de otro producto concentra, en cambio, prácticamente todos los defectos explotables: ningún nonce, manejadores abiertos al público, identificadores de acceso directo sin autorizar y un token privilegiado escrito en el código.

El riesgo estructural no está en ninguno de esos defectos por separado, sino en dos condiciones de fondo: **no hay control de versiones** —lo que impide saber qué está vivo, quién cambió qué y cómo revertir— y **el sistema no es reproducible** sin un clon de la base de datos, lo que deja al equipo sin entorno de pruebas y convierte cada cambio en una intervención sobre producción.

12.2 Recomendaciones

Ordenadas por prioridad. El esfuerzo es una estimación orientativa basada en el volumen de código afectado, no un presupuesto.

Prioridad 1 — Inmediata (riesgo activo sobre datos o ingresos)

#	Acción	Justificación	Esfuerzo	Ref.
1	Exigir sesión y pertenencia en `get_test_results`: usar siempre `get_current_user_id()` y aceptar un `user_id` ajeno solo si el solicitante es el administrador de su empresa. Añadir nonce y retirar el registro `nopriv`.	Hoy cualquiera puede leer los resultados del test de cualquier empleado iterando identificadores. Es el único hallazgo que expone datos personales sensibles sin ninguna credencial.	Bajo	H-1
2	Rotar el token de las APIs `aprobandome/v1` y dejar de sembrarlo desde el código: generarlo al activar el tema con `wp_generate_password()` y exponerlo en una pantalla de ajustes. Eliminar el valor literal de los dos archivos.	El valor actual está en el código: cualquier copia del tema permite crear usuarios con rol de administrador de empresa. Rotarlo requiere además actualizar los consumidores legítimos.	Bajo	H-2
3	Validar en `get_assign_course_ids_to_users` que cada curso solicitado pertenece al itinerario del usuario o ha sido comprado, antes de llamar a `ld_update_course_access()`.	Permite a cualquier empleado con sesión matricularse en todo el catálogo. Afecta directamente a los ingresos por venta de cursos.	Bajo	H-3

#	Acción	Justificación	Esfuerzo	Ref.
4	Verificar si `novamira` y `easy-mcp-ai` están activos en producción y, si lo están, desactivarlos o restringirlos a un entorno de desarrollo.	El primero declara exponer ejecución de PHP arbitrario y estar destinado solo a desarrollo. Es una comprobación de cinco minutos con un desenlace potencialmente crítico.	Mínimo	H-6
5	Trasladar los cuatro secretos del código a constantes de `wp-config.php` u opciones, empezando por el secreto de SSO, y rotarlos de forma coordinada en los tres sitios.	El secreto de SSO permite forjar un token de acceso para cualquier correo en todo el ecosistema. Al estar en el archivo del plugin, viaja en cada copia y cada backup.	Medio	H-4

Prioridad 2 — Corto plazo (reducción de riesgo y de coste de error)

#	Acción	Justificación	Esfuerzo	Ref.
6	Poner el proyecto bajo control de versiones, con un primer commit del estado actual y un `.gitignore` que excluya terceros y traducciones.	Es la acción con mejor relación entre coste y beneficio de toda la lista: habilita revisión, reversión y trazabilidad, y convierte en comprobables buena parte de las inferencias de este informe.	Bajo	10.3
7	Añadir nonce a los 15 manejadores AJAX del tema y retirar `nopriv` de los que exigen sesión (test, cursos, carrito, correos), dejándolo solo donde es imprescindible (acceso y registro).	Cierra la falsificación de petición entre sitios en operaciones con efecto real. El patrón ya está resuelto en los plugins propios: se puede copiar.	Medio	H-5
8	Convertir a `$wpdb->prepare()` las tres consultas de `UserQueries` y `new-test.php`, y sanear `$_POST['answers']` validando su estructura antes de guardarla.	Cierra la inyección SQL y evita que un valor malformado llegue al cálculo de resultados, a la asignación de cursos y al generador externo de PDF.	Bajo	H-7, H-8
9	Paginar o trocear la exportación a Excel y precalcular el progreso: almacenar el porcentaje por usuario y curso, actualizándolo al completar una lección en lugar de recalcularlo en cada listado.	Es el único punto con riesgo real de agotar el tiempo de ejecución, y afecta a una función que el cliente usa de cara a su propia dirección.	Medio	10.1
10	Añadir identificador de un solo uso al token de SSO y registrar su consumo en un transitorio de 60 segundos.	Elimina la reutilización del token dentro de su ventana de validez. Cambio contenido y local al plugin, aunque exige despliegue coordinado en los tres sitios.	Bajo	H-9
11	Retirar de producción `ap-debug.php`, el shortcode `[mi_sso_debug]` y la copia `gestionar-empresa-1.31.0/`. Fijar las versiones de las librerías de CDN y añadir `integrity`.	Limpieza de bajo riesgo que reduce superficie de ataque y ruido al navegar el proyecto.	Mínimo	H-12, H-13

Prioridad 3 — Medio plazo (sostenibilidad)

#	Acción	Justificación	Esfuerzo	Ref.
12	Externalizar los identificadores de `AppConstants` y el curso fijo de `save-test.php` a opciones configurables, y añadir el prefijo de WordPress a `formularios_pagos`.	Es el requisito previo para tener un entorno de preproducción fiable. Mientras los identificadores de producción estén en el código, no hay forma de probar un cambio antes de publicarlo.	Medio	10.3
13	Unificar las cuatro rutas de alta de usuario en un único servicio interno que concentre cupo, credenciales, propagación y notificación, y hacer que los cuatro puntos de entrada lo invoquen.	Hoy un cambio en la regla de cupo o en la propagación hay que replicarlo en cuatro sitios, con el riesgo de que uno quede atrás. Es la duplicación más costosa del proyecto.	Alto	10.2
14	Dar fiabilidad a la propagación: encolar las cuatro llamadas salientes, reintentarlas y registrar su resultado, en lugar de ejecutarlas en línea dentro de la petición del usuario.	Elimina los estados incoherentes entre los tres sitios y desacopla la experiencia del cliente de la latencia de terceros.	Medio	6.1
15	Mover la lógica de `thankyou.php` a un hook del ciclo de vida del pedido (`woocommerce_payment_complete` o equivalente).	Evita el trabajo y los correos repetidos al recargar la página y permite reprocesar un pedido sin depender de que el cliente abra esa URL.	Bajo	10.1
16	Definir capacidades propias (por ejemplo `ge_manage_companies`) y autorizar por capacidad en lugar de por nombre de rol. Corregir la incoherencia entre el ajuste de rol y la constante de `AP_Security`.	Permite delegar la gestión de empresas sin conceder el control total del sitio y hace predecible el comportamiento con roles múltiples.	Medio	H-10, 8.4
17	Decidir y documentar qué código del tema está vivo: retirar la funcionalidad de test «en pareja» y «de padres», los shortcodes `sistemadi_*` y las referencias a `aprobandome.com` que ya no se usan.	Reduce en un porcentaje apreciable el volumen de código a entender y mantener. Debe hacerse **después** del punto 6, para poder revertir con seguridad.	Medio	10.3
18	Documentar el esquema de `wp_user_members` e incorporar su creación al plugin que la usa, con versionado de esquema que sustituya a los `SHOW TABLES` por petición.	Hoy es un punto ciego: ningún módulo la crea pero seis la usan. Sin esto, una instalación nueva no arranca.	Bajo	5.1, 11.4
19	Automatizar los recordatorios y la caducidad de membresías con WP-Cron, y añadir pruebas automatizadas al menos para el cálculo de resultados y la asignación de cursos.	El negocio depende hoy de que una persona pulse un botón, y la pieza con más reglas del sistema no tiene ninguna red de seguridad frente a cambios.	Medio	8.5, 10.3

#	Acción	Justificación	Esfuerzo	Ref.
20	Completar el plan de seguridad ya iniciado en `mu-plugins`: pasar la CSP a modo bloqueo tras revisar los informes recogidos y activar HSTS.	El trabajo está hecho y documentado por fases; solo falta ejecutar las dos últimas. Aprovecha una inversión ya realizada.	Bajo	9.1

12.3 Verificaciones pendientes

Estas cuestiones no se pudieron resolver con el material disponible y conviene cerrarlas antes de tomar decisiones basadas en este informe:

Cuestión	Cómo resolverla
Qué plugins están realmente activos, en especial `novamira`, `easy-mcp-ai`, Watu y WatuPRO.	Opción `active_plugins` de la base de datos, o la pantalla de plugins.
Esquema e índices reales de `wp_user_members` y `formularios_pagos`, y qué componente las creó.	`SHOW CREATE TABLE` sobre ambas.
Versiones instaladas de WordPress, PHP y de los plugins de terceros frente a sus avisos de seguridad conocidos.	Panel de WordPress y `phpinfo()`. Este informe no incluye análisis de vulnerabilidades conocidas de terceros.
Si existen tareas programadas fuera del código y si WP-Cron está operativo.	Inspección de `cron` en las opciones y del cron del sistema.
Tipo de servidor web, para saber si `.htaccess` o `web.config` protegen realmente `novamira-sandbox/`.	Configuración del alojamiento.
Qué registran los servicios externos sobre las contraseñas que reciben en los webhooks de alta.	Consulta al responsable de `api.efrenmartinezortiz-ia.com`.
Qué parte del código heredado del tema está en uso real.	Trazas de acceso del servidor, o instrumentación temporal de los manejadores sospechosos.

> **Nota final sobre el orden de trabajo**
>
> Las cinco acciones de prioridad 1 son todas de esfuerzo bajo o mínimo y abordan riesgos explotables con una petición HTTP o con una copia del código. Pueden ejecutarse en pocos días. La recomendación número 6 —control de versiones— debería acompañarlas: sin ella, cada corrección posterior se aplica sin red.